

AI COMPETITIVE INTELLIGENCE DIGEST

End-to-end technical documentation

An n8n workflow that monitors six AI-lab competitors of OpenAI across four public data sources, analyses each new story with an LLM acting as a competitive analyst, and delivers a single priority-ranked HTML briefing by email every morning.

39 nodes · 4 data sources · 1 workflow · Gemini 2.5 Flash

Live Google Sheet:

https://docs.google.com/spreadsheets/d/1fMCeM_tuTnzR05-aVOlvh5zwd5SuhHoA1209xRKOEys/edit

1. What it does and why

The problem

Competitive intelligence is a real job that mostly gets done badly. Someone on the strategy team opens fifteen tabs on Monday morning, skims headlines, and forms an impression. It is not repeatable, it is not recorded, and the same story read by two people produces two different conclusions.

The tedious parts are all automatable: gathering the sources, discarding what has already been seen, and writing a first-pass assessment. The judgement part is not. This workflow does the first three and hands a structured draft to a human.

What it does

Every morning at 09:00 the workflow performs seven steps:

1. Checks four public sources for each of six competitors of OpenAI.
2. Normalises four unrelated payload shapes into one common schema.
3. Discards anything already reported on in a previous run.
4. Sends each genuinely new story to Gemini, prompted to read it as an analyst inside OpenAI.
5. Assigns a priority using deterministic rules rather than the model's opinion.
6. Writes every story to Google Sheets for the historical record.
7. Emails one HTML briefing, grouped Critical, High, Medium and Low.

Who is monitored, and why

Competitor	Rationale for inclusion
Anthropic	Claude — closest rival on frontier model quality and enterprise API business
Google DeepMind	Gemini — backed by Google distribution and compute
xAI	Grok — bundled into X and increasingly into other products
Perplexity	AI search — competes for the consumer question-answering use case
Mistral AI	Open-weight European models — competes on cost and sovereignty
Cohere	Enterprise and RAG-focused — competes for the same business customers

The framing is not decorative

The phrase "You are a Competitive Intelligence Analyst embedded in OpenAI's strategy team" is literally in the Gemini system prompt. That is why the analysis reads "this threatens our pricing power" rather than producing a neutral summary.

Every story is judged against one question: does this affect our revenue, our technical lead, or our customers?

Data sources

Source	Endpoint	What it contributes
Company blog	RSS feed (Google News site: filter where no feed exists)	Official announcements
GitHub	api.github.com/orgs/{org}/repos	Repository activity, shipping signals
Hacker News	hn.algolia.com/api/v1/search_by_date	Developer sentiment, early reaction
Google News	news.google.com/rss/search	Third-party coverage and analysis

2. Architecture

The workflow is a single n8n file organised into four stages. Stage 1 runs inside a loop, once per competitor. Stages 2 to 4 run exactly once, on the loop's "done" output, with every competitor's items pooled together.

```
09:00 cron
|
v
Competitor Registry (6 organisations)
|
v
+-- Loop Over Competitors <-----+
| (batch size 1, "loop" output)   |
v                                  |
Blog RSS | GitHub | Hacker News | Google News
|         |         |         |         |
+-----+-----+-----+-----+
|         |         |         |         |
v                                  |
Merge All Sources (4 inputs)      |
v                                  |
Normalize To Common Schema        |
v                                  |
Fetch Succeeded? --false--> error_log
| true                                     |
+--> Throttle (2s) -----+
|
("done" output, fires once)
v
Dedupe & Sort
v
Read Seen Ledger (Sheets)
v
Filter Already Seen
v
Limit (10, testing cap)
v
AI Competitive Analyst (Agent)
+ Google Gemini Chat Model
+ Structured Output Parser
v
Validate AI Output
v
Route By Priority (Switch)
Critical | High | Medium | Low
v
Merge (4 inputs)
v
Append To History Sheet
v
Append To Seen Ledger
v
Aggregate For Report
v
Build HTML Report
v
Send Email (Gmail)
```

3. Node-by-node walkthrough

Stage 1 — Collect

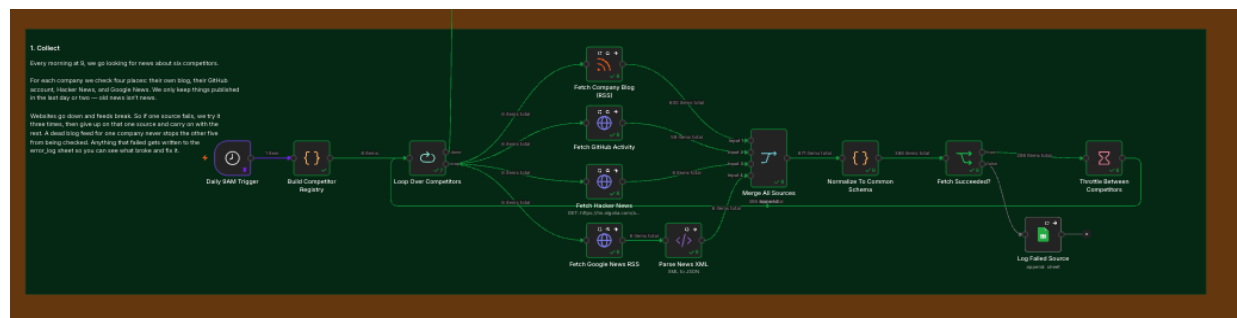


Figure 1. The collection stage. One loop iteration per competitor, four parallel fetches, a merge, a normaliser, and a handled error branch into the error_log sheet.

Node	Type	Function
Daily 9AM Trigger	Schedule Trigger	Fires at 09:00 daily.
Build Competitor Registry	Code	Emits one item per competitor with company, github_org, news_query and blog_rss, plus run-wide constants LOOKBACK_HOURS (48), MAX_ITEMS_PER_RUN and a precomputed cutoff_epoch. This is the only node edited to add or remove a competitor.
Loop Over Competitors	Split In Batches	Batch size 1. The "loop" output drives the fetches; the "done" output fires once at the end carrying all competitors' items pooled.
Fetch Company Blog (RSS)	RSS Feed Read	Official announcements from the company feed.
Fetch GitHub Activity	HTTP Request	GET /orgs/{org}/repos sorted by pushed date, 10 per page.
Fetch Hacker News	HTTP Request	Algolia search_by_date, filtered server-side with created_at_i > cutoff_epoch.
Fetch Google News RSS	HTTP Request	Publisher search feed with when:2d, fetched as raw text.
Parse News XML	XML	Converts the Google News XML response into JSON.
Merge All Sources	Merge (4 inputs)	Pools four heterogeneous payloads into a single stream.
Normalize To Common Schema	Code	Detects source type by payload shape and maps everything to one schema. Applies the 48-hour lookback window.
Fetch Succeeded?	IF	Routes items tagged _fetch_error away from the happy path.
Log Failed Source	Google Sheets	Appends to error_log and deliberately dead-ends.
Throttle Between Competitors	Wait (2s)	Politeness delay, then feeds back into the loop.

Error handling is the point of this stage

Every fetch node carries the following settings:

```
retryOnFail:    true
maxTries:       3
```

```
waitBetweenTries: 2000    (2 seconds, escalating)
onError:             continueRegularOutput
alwaysOutputData: true
```

A failed node emits an error object instead of throwing. The normaliser detects that shape, tags it, and the IF node routes it to the error log. One dead feed never stops the other five competitors.

A wiring rule that cost real debugging time

Log Failed Source must NOT connect back to Throttle Between Competitors.

If it does, the Wait node has two inbound edges and therefore executes twice per iteration, pushing two signals back into the loop. The loop is then called more times than it has competitors, and each extra call after the list is exhausted fires the "done" output again — producing duplicate emails and duplicate sheet rows.

Error branches should terminate, not rejoin the main path.

Stage 2 — Dedupe and idempotency

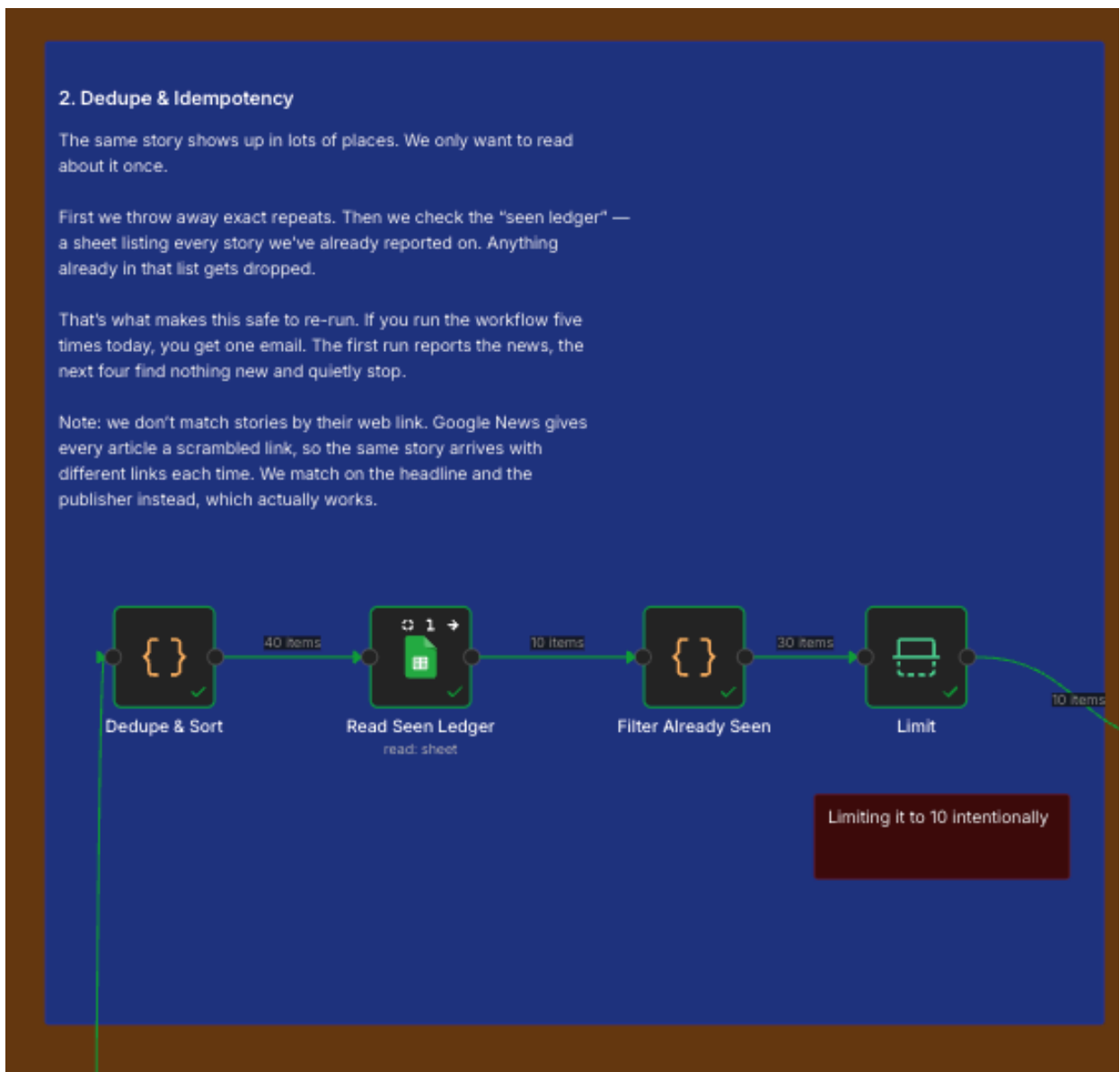


Figure 2. The deduplication and idempotency gate. Runs once, on the loop's done output, with all competitors pooled. The Limit node is capped at 10 intentionally.

Node	Type	Function
Dedupe & Sort	Code	Runs once on the loop's done output. Drops error and empty items, collapses exact fingerprint duplicates, sorts newest first, caps the batch at MAX_ITEMS_PER_RUN.
Read Seen Ledger	Google Sheets	Reads every fingerprint previously reported. executeOnce is set to true — without it the node fires once per item and multiplies the entire downstream chain.
Filter Already Seen	Code	Anti-joins candidates against the ledger. Emits a single _no_new_items sentinel when nothing survives so the flow terminates cleanly rather than stalling.
Limit	Limit	maxItems is set to 10. This is a deliberate cost cap — see below.

The limit of 10 is intentional

The Limit node is capped at 10 items on purpose. It is the last gate before the LLM, so it acts as a direct dial on API cost: 10 items means exactly 10 Gemini calls per run.

This keeps test and demonstration runs cheap while the workflow is being tuned. For production use, raise it to 40 — or delete the node entirely and let MAX_ITEMS_PER_RUN in the registry govern instead, since it performs the same function one node earlier.

Nothing else in the workflow depends on this value. It exists purely as a cost guard.

Why fingerprints rather than URLs

The obvious idempotency key is the article URL. It does not work here.

Google News returns opaque redirect tokens of the form `news.google.com/rss/articles/CBM...` rather than publisher URLs. The same story arriving via two feeds carries two different tokens, so a URL-keyed ledger lets both through and the same story is reported twice.

The fingerprint used instead is:

```
djb2( company + "|" + normalized_title + "|" + publisher_host )
```

This is deterministic, requires no crypto import, and survives both redirect tokens and minor headline edits.

Stage 3 — Analyse

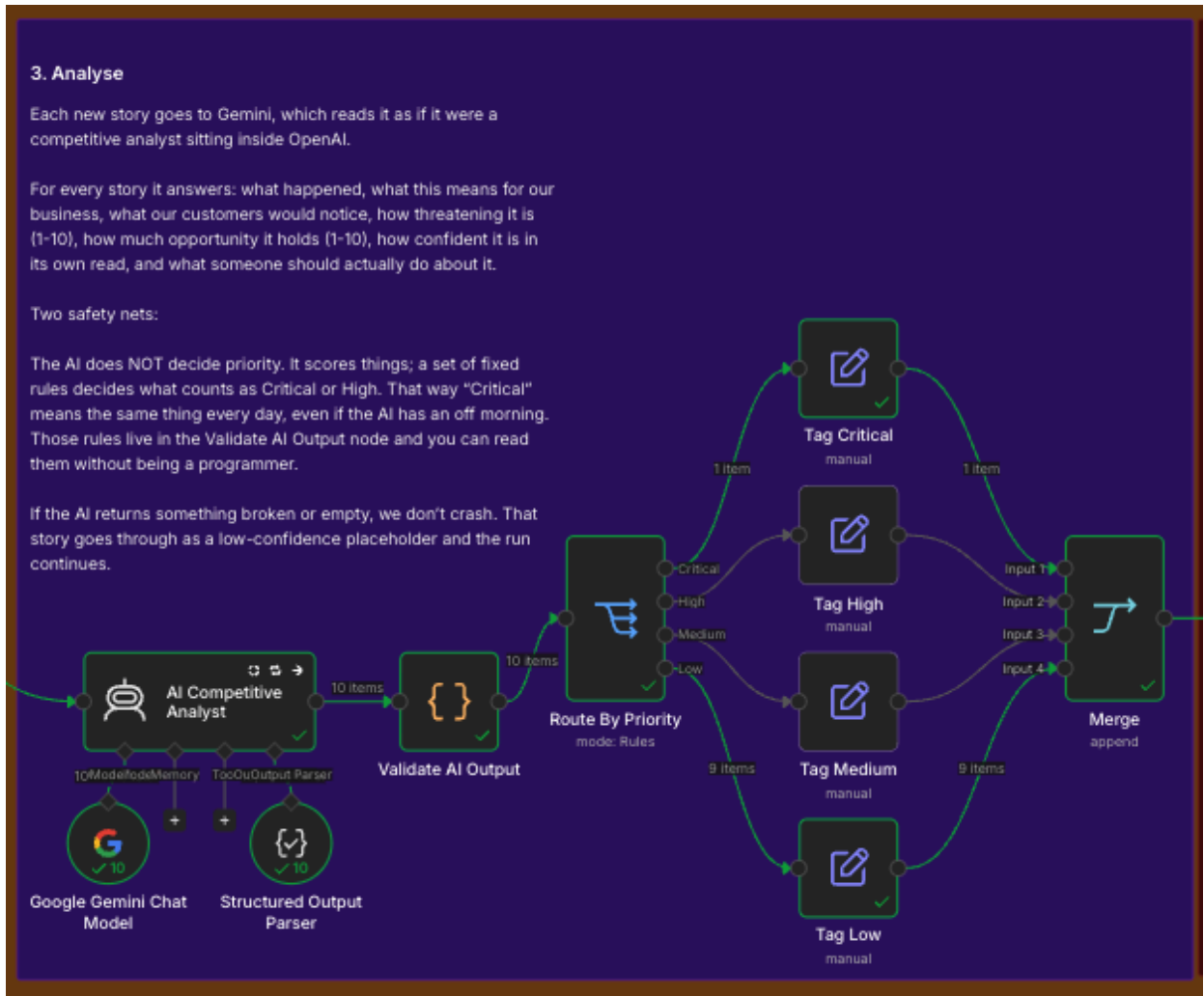


Figure 3. The analysis stage. Gemini scores each story; deterministic rules assign priority; the Switch routes into four branches which the Merge node reconverges into one batch.

Node	Type	Function
AI Competitive Analyst	AI Agent	System prompt frames the model as an analyst inside OpenAI. The user message carries the normalised article.
Google Gemini Chat Model	Gemini Chat Model	Temperature 0.2 — low, because this is assessment rather than creative writing.
Structured Output Parser	Output Parser	Enforces the JSON response shape via a jsonSchemaExample.
Validate AI Output	Code	Repairs malformed responses and applies the deterministic priority rules engine.
Route By Priority	Switch (4 outputs)	Critical, High, Medium and Low, with fallback to Low.
Tag Critical / High / Medium / Low	Set (x4)	Attach priority_colour and priority_rank for report rendering.
Merge	Merge (4 inputs)	Reconverges the four priority branches into a single batch.

What the model returns

```
{
  "category": "model_launch",
  "summary": "Two to three sentences on what happened.",
  "business_impact": "Effect on OpenAI revenue, moat, or roadmap.",
  "customer_impact": "What an OpenAI customer would notice.",
  "sentiment": "negative",
  "threat_level": 7,
  "opportunity_level": 3,
  "priority": "High",
  "recommended_action": "A concrete, assignable next step.",
  "confidence": 8,
  "executive_summary": "One sentence under 25 words."
}
```

Safety net one: the AI does not decide priority

The model scores threat, opportunity, confidence and sentiment. A set of fixed regular-expression rules in Validate AI Output decides what counts as Critical or High:

Detected signal	Assigned priority
Funding round or acquisition	Medium
Model launch or release	High
Pricing change	High
Hiring surge	High
Launch or pricing signal, and model threat score of 7 or above	Critical

This means "Critical" carries the same meaning every day even if the model has an off morning. It also means the business logic can be read and challenged by someone who does not write code.

Safety net two: a broken response never crashes the run

Validate AI Output strips markdown code fences, attempts a brace-match rescue on partial JSON, and falls back to a low-confidence placeholder row if parsing fails entirely. The run continues and the failure is visible in the output rather than silent.

Why the Merge node is not optional

n8n executes a node once per inbound connection. Four Tag nodes wired directly into one Sheets node would cause every node downstream — including the email — to run four times, producing four separate emails and four sets of sheet rows.

The Merge node reconverges the branches into a single batch. This is precisely why the workflow delivers one consolidated email rather than one per priority level.

Stage 4 — Deliver

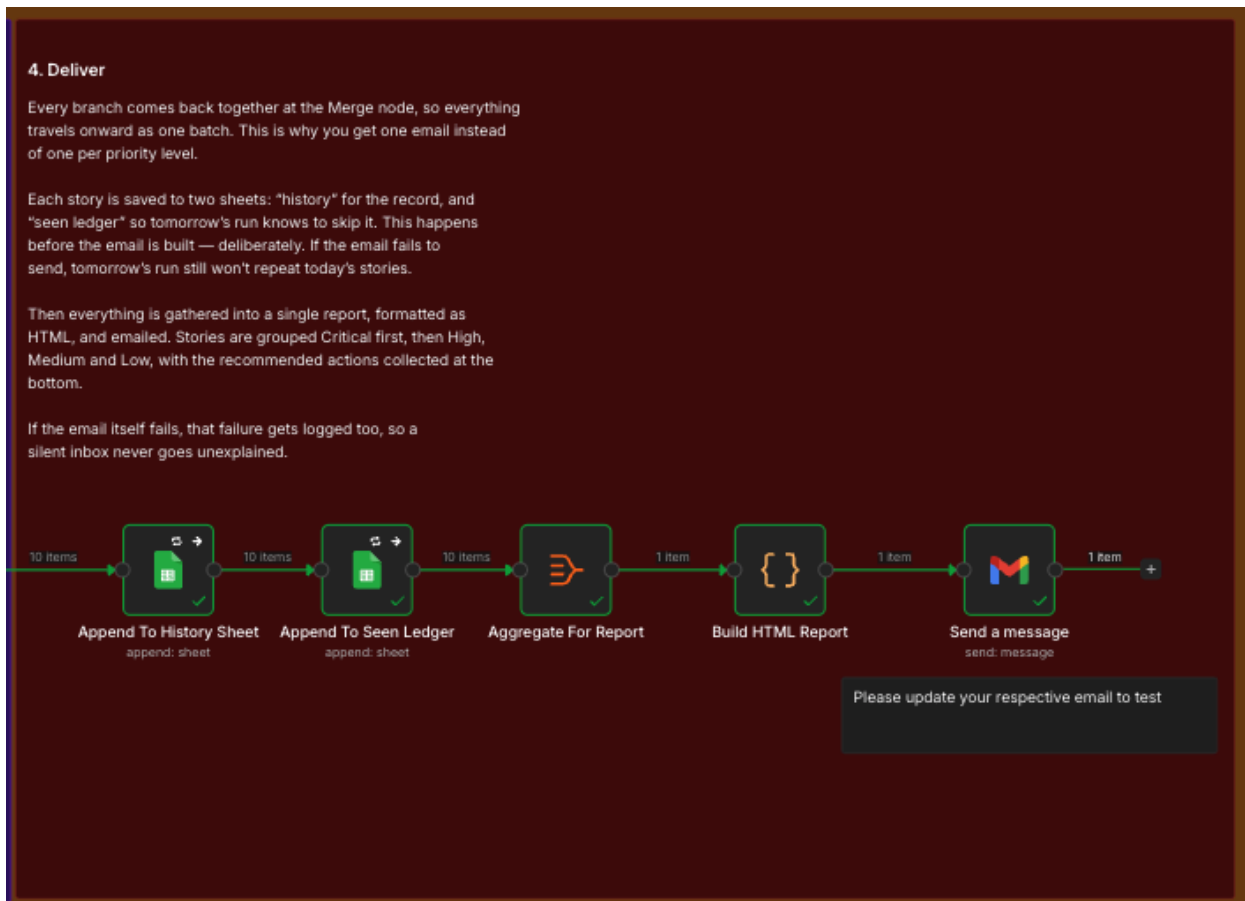


Figure 4. The delivery stage. Sheets are written before the email is built, then all items are aggregated into a single report and sent.

Node	Type	Function
Append To History Sheet	Google Sheets	One row per story — the analytical record.
Append To Seen Ledger	Google Sheets	Fingerprint and URL — the idempotency record.
Aggregate For Report	Aggregate	aggregateAllItemData collapses every item into a single items array on one output item.
Build HTML Report	Code	Renders the email. Inline styles and table layout only, because Gmail and Outlook strip style blocks.
Send a message	Gmail	Delivers the HTML briefing.

Write before send is deliberate

Both sheets are written before the email is built. If delivery fails, the following day's run still will not repeat today's stories, because the ledger already contains them.

The tradeoff is explicit: a delivery failure loses that day's email entirely. This is the correct trade. A missed email is visible and recoverable from the history sheet, whereas silently re-reporting the same twelve stories every morning until somebody notices is not.

Live output sheet:

https://docs.google.com/spreadsheets/d/1fMCeM_tuTnzR05-aVOlvh5zwd5SuhHoA1209xRKOEys/edit

4. The delivered output

This is what actually lands in the inbox. Everything described above exists to produce this single artefact.

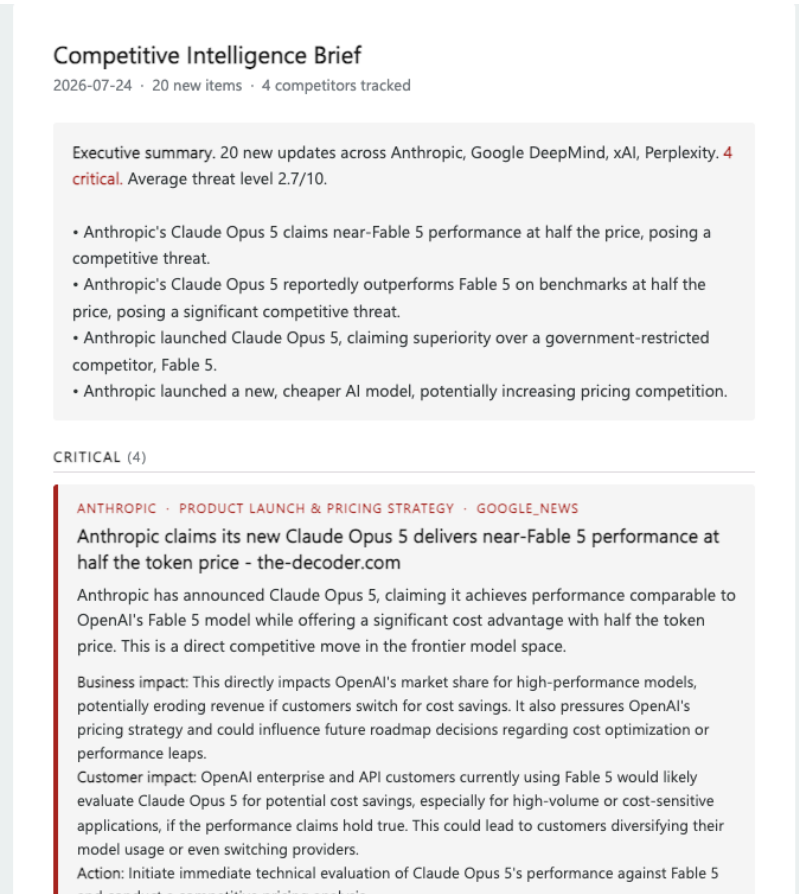


Figure 5. A real delivered briefing: 20 new items across 4 competitors, 4 scored Critical, average threat 2.7 out of 10.

Layout, top to bottom

1. HEADER
Competitive Intelligence Brief
2026-07-24 · 20 new items · 4 competitors
2. EXECUTIVE SUMMARY (grey card)
Counts · critical count in red · avg threat
- One-line summary per critical item
3. CRITICAL (4)
COMPANY · CATEGORY · SOURCE (coloured)
Headline - hyperlinked to the article
Summary (2-3 sentences)
Business impact: ...
Customer impact: ...
Action: ...
Threat 8/10 · Opp 1/10 · Impact 5.2 ·
Confidence 6/10 · Sentiment negative
4. HIGH PRIORITY / MEDIUM / LOW
Same card format, different accent colour

5. RECOMMENDED ACTIONS
Bulleted, drawn from Critical + High items only
6. FOOTER - sources used, scoring caveat

The reasoning behind each section

Section	Purpose and design reasoning
Header	Date, item count and competitor count. Tells the reader in one second whether this is a busy day or a quiet one.
Executive summary	The only part most readers will finish. Counts, the critical count highlighted in red, average threat level, then one bullet per critical item. Each bullet is the model's executive_summary field, constrained by the prompt to a single sentence under 25 words.
Priority sections	One per level, ordered Critical, High, Medium, Low. Empty sections are omitted entirely, so a quiet day produces a short email rather than four empty headers. The item count appears beside each header.
Story cards	Each carries a coloured left border matching its priority, set by the four Tag nodes in Stage 3. The three impact lines sit in a grey sub-panel so they read as analysis rather than as more description.
Score strip	Threat, opportunity, impact, confidence and sentiment in small grey type at the foot of each card. Deliberately understated: these are advisory model outputs, not facts, and should not compete visually with the summary. If the LLM response failed to parse, an "AI parse degraded" marker appears here in red.
Recommended actions	Every action from Critical and High items collected into one list. This is the section a strategy lead can forward without reading anything else.

Priority colour coding

Priority	Border colour	Meaning
Critical	#b3261e (red)	A launch or pricing event with a model threat score of 7 or above
High	#c77700 (amber)	Model launch, pricing change, or hiring surge
Medium	#1a73e8 (blue)	Funding round or acquisition
Low	#5f6368 (grey)	Everything else — routine coverage

Subject line

The subject escalates automatically based on content:

Competitive Brief 2026-07-24 - 20 updates
[CRITICAL] Competitive Brief 2026-07-24 - 4 critical, 20 updates

The [CRITICAL] prefix appears only when at least one item clears the Critical rule, so it remains meaningful as an inbox filter.

Rendering constraints

The report is built in Build HTML Report as a single HTML string, under two hard constraints imposed by email clients:

Constraint	Reason
Inline styles only — no style block	Gmail and Outlook strip head styles entirely.
Table-based layout — no flexbox or grid	Outlook's rendering engine is Word, which supports neither.

Every value is HTML-escaped before insertion, so a headline containing an ampersand or angle bracket cannot break the layout.

Reading the sample above honestly

Two things visible in this real run are worth naming rather than hiding:

- Four near-identical Claude Opus 5 stories. Four outlets covered one launch, producing four fingerprints and four separate cards. This is the known clustering limitation in section 10 — a title-similarity pass in Dedupe & Sort would collapse these into a single card with citations.
- Only four of six competitors appear. Two produced nothing on that run. Check `error_log` before assuming it was a quiet news day — a dead feed looks identical to silence from the reader's side, which is exactly why that sheet exists.

5. The data contract

Four sources with four unrelated payload shapes converge on one schema. Everything downstream depends only on this contract, which is why adding a fifth source requires changing exactly one node.

```
{
  company:      "Anthropic",
  source:       "google_news",    // company_blog | github |
                                   // hacker_news | google_news
  category:     "news",
  title:        "...",
  date:         "2026-07-24T09:12:00.000Z",  // ISO 8601 always
  url:          "https://...",
  content:      "...",           // HTML stripped, 2500 char cap
  fingerprint: "anthropic-3f9a2c11",
  run_date:     "2026-07-24",
  collected_at: "2026-07-24T09:00:04.812Z",
  _fetch_error: false
}
```

Failed fetches use a parallel shape so the IF node can route them without special-casing:

```
{
  company:      "Cohere",
  source:       "upstream_fetch",
  _fetch_error: true,
  error_message: "Feed not recognized as RSS 1 or 2.",
  run_date:     "2026-07-24",
  collected_at: "2026-07-24T19:42:07.213Z"
}
```

The normaliser detects source type by payload shape rather than by node name:

Detection test	Identified source
Array.isArray(d.hits)	Hacker News — one object containing an array
d.rss && d.rss.channel	Google News — parsed XML
d.full_name && d.html_url	GitHub — array already split into items by n8n
d.link && (d.isoDate d.pubDate)	RSS blog feed

Every branch is wrapped in try/catch, so a parse failure becomes a tagged error item rather than an exception.

6. Setup and credentials

Prerequisites

- n8n, cloud or self-hosted, version 1.60 or later for AI Agent v2.1
- A Google account with access to Sheets and Gmail
- A Google Gemini API key

Credentials

Credential	Type in n8n	Used by
Gemini	Google Gemini(PaLM) Api	Google Gemini Chat Model
Google Sheets	Google Sheets OAuth2 API	All four Sheets nodes
Gmail	Gmail OAuth2 API	Send a message

Never commit real secrets

n8n exports credential references — IDs and display names — rather than secrets, so no API key is present in the workflow file.

The spreadsheet ID and recipient email address are present, however. The spreadsheet below is shared read-only on purpose so that reviewers can inspect the output. If you fork this workflow for your own use, create your own spreadsheet and replace the documentId in all four Sheets nodes, and the sendTo address in the Gmail node.

Google Sheet — three tabs required

Live sheet used by this workflow:

https://docs.google.com/spreadsheets/d/1fMCeM_tuTnzR05-aVOlvh5zwd5SuhHoA1209xRKOEys/edit

Share setting should be "Anyone with the link — Viewer" so reviewers can see the output without requesting access.

Create a single spreadsheet with three tabs. Header row names must match exactly.

Tab: history

```
run_date | company | category | priority | threat_level |  
opportunity_level | impact_score | confidence | sentiment |  
source | url | summary | recommended_action
```

Tab: seen_ledger

```
fingerprint | run_date | company | url
```

Tab: error_log

```
timestamp | run_date | company | source | error_message
```

Import and configure

8. In n8n choose Workflows, then Import from File, and select workflow.json.

9. Open each Google Sheets node and select your spreadsheet and the matching tab.
10. Open Google Gemini Chat Model, select your Gemini credential, and set the model to `models/gemini-2.5-flash`.
11. Open Send a message, select your Gmail credential, and set the recipient address.
12. Optionally open Build Competitor Registry and edit the `COMPETITORS` array.

A note on RSS feeds

Most AI laboratories do not publish RSS feeds. Anthropic, xAI, Mistral, Cohere and Perplexity all lack one, and Perplexity additionally blocks non-browser requests with an HTTP 403. Only Google DeepMind offers a working official feed.

The registry therefore uses Google News publisher-filtered feeds as a stand-in, in the form:

```
https://news.google.com/rss/search?q=site:anthropic.com  
&hl=en-US&gl=US&ceid=US:en
```

These are always available, require no credentials, and introduce no dependency on volunteer-maintained mirrors. The tradeoff is headlines and snippets rather than full post bodies, which is why confidence scores on those items run lower and why the priority rules cap thin-sourced items below Critical.

7. How to run it

Workflow URL: <https://aditya-mb.app.n8n.cloud/workflow/27Olz4WrPwAA8wBY>

Login Credentials:

Email: mbaditya1909@gmail.com

Password: Aditya@123

Manual execution, recommended first

Open the workflow and choose Execute Workflow. A run takes one to three minutes depending on how many items pass the ledger.

The first run analyses everything within the 48-hour window and is therefore the most expensive. Subsequent runs see only genuinely new stories.

Scheduled execution

Toggle the workflow to Active. It will fire daily at 09:00 in the instance timezone.

Cost control before activating

Setting	Location	Testing	Production
maxItems	Limit node	10	40, or delete the node
MAX_ITEMS_PER_RUN	Build Competitor Registry	40	40
retryOnFail	AI Competitive Analyst	off	on

Whatever number sits in the Limit node is very close to what is billed to Gemini per run. The workflow ships with that value set to 10 deliberately.

8. How to verify it worked

Run the workflow manually, then check five things in order.

Output sheet to inspect:

https://docs.google.com/spreadsheets/d/1fMCeM_tuTnzR05-aVOlvh5zwd5SuhHoA1209xRKOEys/edit

1. Every node after the loop executed exactly once

Watch the execution badges on the canvas. From Dedupe & Sort onward, every node must read a count of 1.

If any node after the loop shows 2 or higher, something is fanning in and the result will be duplicate emails and duplicate sheet rows. The two usual causes are Read Seen Ledger missing its executeOnce setting, and Log Failed Source being wired back into Throttle Between Competitors.

2. One email arrives

The subject line should read similar to:

Competitive Brief 2026-07-24 - 20 updates

Open it. Sections should appear in priority order with empty ones hidden, headlines should be clickable, and the Recommended Actions block should be populated at the bottom. Section 4 of this document shows a screenshot of a successful run alongside a breakdown of every section.

3. The history tab has one row per story

Every row should carry a priority, threat_level, opportunity_level and summary. A row whose summary begins with "[AI analysis unavailable]" means the model returned something unparseable and the fallback did its job — the run survived, which is the intended behaviour.

4. The seen_ledger tab grew by the same count

The number of new rows should match the history tab. If the two differ, one of the two Sheets appends is failing; check error_log.

5. Idempotency — the important check

Run the workflow a second time immediately

Expected result: the execution reaches Filter Already Seen, finds nothing new, and stops. No second email is sent and no new sheet rows are created.

This is the single best proof that the workflow is safe to run in production. If the history tab grows on the second run, the ledger read is failing.

Bonus check: prove the error handling

Open Build Competitor Registry and deliberately break one feed URL — for example, change the Cohere blog_rss value to a path that returns 404. Re-run the workflow.

Expected result: the workflow completes, the other five competitors still report, and error_log gains a row.

timestamp	company	source	error_message
2026-07-24T19:42:07.213Z	Cohere	upstream_fetch	Feed not recognized as RSS 1 or 2.
2026-07-24T19:43:47.007Z	Mistral AI	upstream_fetch	Request failed with status code 404
2026-07-24T19:43:37.991Z	Perplexity	upstream_fetch	Request failed with status code 403
2026-07-24T19:43:29.956Z	xAI	upstream_fetch	Request failed with status code 404

The table above is a genuine excerpt from a real run. Five sources failed that morning and the briefing was still delivered.

9. Design decisions

Decision	Reasoning
Priority is rule-based, not model-decided	The LLM scores; fixed rules classify. Consistency across runs matters more than nuance, and a non-engineer can read and challenge a table of rules in a way they cannot challenge a prompt.
Fingerprints rather than URLs	Google News redirect tokens make URL-keyed deduplication silently leaky. This only surfaces by running the workflow and observing the same story arrive twice.
Error branches terminate	Failed items go to the log and stop. Rejoining them to the main path caused the loop to over-fire and produce duplicate emails — a subtle bug that presented as an Aggregate problem and was not one.
Sheets written before email	A delivery failure must not corrupt idempotency state.
One registry node	Adding a competitor is a one-line edit in a single Code node. Every other node reads generically from the item.
Detection by payload shape	The normaliser does not care which node produced an item, so reordering or adding sources does not break it.

10. Known limitations

Limitation	Detail and mitigation
RSS feeds drift	AI laboratories restructure their newsrooms frequently. When a competitor stops appearing, check <code>error_log</code> first — that is exactly what it exists for.
GitHub rate limits	The unauthenticated API permits 60 requests per hour. Six competitors daily is comfortable; add a GitHub credential before increasing frequency.
Google News is deep but stale	Median item age skews several days and feeds cap at roughly 100 items with no pagination. Freshness comes from the blog, GitHub and Hacker News sources; Google News widens the net.
Reddit is deliberately absent	Unauthenticated <code>.json</code> endpoints were deprecated in May 2026 and now return 403. Adding Reddit would require an approval-gated OAuth application.
Near-duplicate stories get separate cards	Four outlets covering one launch produced four separate cards in the sample briefing in section 4. A title-similarity clustering pass in Dedupe & Sort would collapse these, and is the most worthwhile next improvement.
No sub-workflow	AI Competitive Analyst and Validate AI Output form a clean seam — one article in, structured JSON out — and would extract into a reusable sub-workflow with no other changes required.